[Bug Report] RSD Processor Does Not Handle State Changes in Floating-Point Registers

Metainformation

JVN-ID: PENDING

Contact: Feel free to reach out for more details to jv.rajendran@tamu.edu

Discoverers: Chen Chen, Jayden Koh, Stephen Muttathil, Jeyavijayan Rajendran, Texas A&M University; Ahmad-Reza Sadeghi, Technische Universitat Darmstadt;

References:
https://github.com/rsd-devel/rsd/issues/103

Proposed CVSS:
CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

CVSS v4.0 Score: **9.3 / Critical**

Macro vector: 000200
Exploitability: High
Complexity: High
Vulnerable system: High
Subsequent system: Low
Exploitation: High
Security requirements: High

Relevant CWEs:
CWE-440: Expected Behavior Violation
CWE-1281: Sequence of Processor Instructions Leads to Unexpected Behavior

Vulnerability Information

Vulnerability type: This vulnerability has implications for incorrect privilege-level transitions and context switching on the RSD processor.

Vendors of the product: RSD is maintained by Ryota Shioya, The University of Tokyo; Koji Inoue, Kyushu University, and the National Institute of Informatics (Japan), Masahiro Goshima.

Product:
RISC-V Out-Of-Order Superscalar Processor (RSD)
https://github.com/rsd-devel/rsd
commit: 7b65f6b

Has the vendor confirmed or acknowledged the vulnerability? No

Attack Type: Remote

Impact: Escalation of Privileges, Denial of Service, Information Disclosure

Affected Components: Software programs utilizing floating-point registers on the RSD processor.

Attack Vectors: Any software program that contains write operations to floating-point registers.

Description of the vulnerability: Executing floating-point write instructions does not upgrade the FS flag in the mstatus register from "Initial" to "Dirty".

Additional Information:

This vulnerability occurs in the handling of floating-point registers by the RISC-V Out-Of-Order Superscalar Processor (RSD), which allows arbitrary processes to perform floating-point operations with data outside of the process's context. The GitHub issue describing the vulnerability can be found here.

Technical Information

This vulnerability enables unprivileged users to gain improper access to floating-point resources through corrupted states created during context switching, cross-process leakage, and improper register saving and restoring.

Background

According to the RISC-V manual:

> "When the status is Dirty, the corresponding state has potentially been modified since the last context save. The status fields will also be updated during execution of instructions, regardless of privilege mode."

RSD GitHub Commit: `7b65f6b`

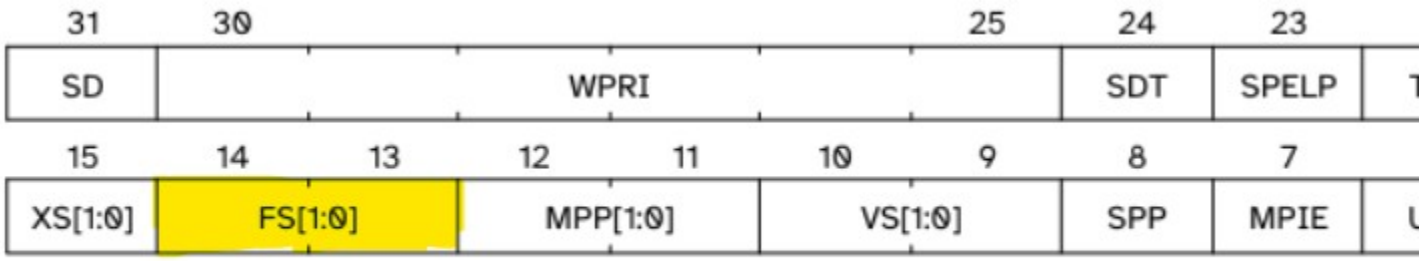


Figure 7. Machine-mode status (mstatu

The specifications for the mstatus register show that the FS flag is in the `mstatus[14:13]`.

Table 11. Encoding of FS[1:0], VS[1:0], and XS[1:0] status fields

Status	FS and VS Meaning	XS Meaning
0	Off	All off
1	Initial	None dirty or clean, some on
2	Clean	None dirty, some clean
3	Dirty	Some dirty

The FS flag in the mstatus register should be modified to correctly reflect the state of the floating-point registers. When none of the floating-point registers are modified and read from, the state should be "Initial'. If a floating-point register has been written to, the state should be "Dirty".

In an example program (provided below as exploit_folder.zip), an attempt is made to write to a floating-point register with the FS state set at "Initial".

```
1b0c:       30029073                csrw    mstatus,t0
1b10:       00000013                nop
1b14:       00000013                nop
1b18:       00000013                nop
1b1c:       3f800537                lui     a0,0x3f800
1b20:       f00500d3                fmv.w.x ft1,a0
```

Figure 1: *Disassembly of the exploit program with relevant instructions.*

The instruction at 0x1b20 performs a write operation to a floating-point register (ft1).

Expected Behavior

If a floating-point register is being written to, the state of the FS flag needs to change from any state to “Dirty”.

Actual Behavior

Below is the waveform describing the processor not properly handling the change in floating-point register states. `regData[33]` is referring to the floating-point register `ft1`, which is being written to; however, the mstatus value remains in the same state, "Initial". This shows that FS is being improperly handled as it does not reflect the correct state of the floating-point registers.



Figure 2: *Waveform of the RSD processor executing FP write instructions without updating the FS flag.*

Steps to Replicate

1. Compiled C code to hex executable. The code used in the screenshot has been attached below.

exploit_folder.zip

2. Run the executable with this line:

vsim ../Project/ModelSim/Main.TestMain +MAX_TEST_CYCLES=50000 +TEST_CODE=exploit_folder/ +ENABLE_PC_GOAL=1 +SHOW_SERIAL_OUT=1 -suppress 8233 +autofindloop +RSD_LOG_FILE=RSD.log +DEBUG_LOG_FILE=Debug.log +REG_CSV_FILE=Register.csv

Proof-of-Concept Exploit

1. In Group 1, set the FS flag in mstatus to "Initial" (01), meaning no floating-point register has been written to in this context.
2. In Group 2, NOP instructions allow the processor enough time to commit the instructions.
3. In Group 3, an instruction writes to a floating-point register. This should change the FS flag from "Initial" to "Dirty" because a floating-point register is being directly modified.
4. In Group 4, NOP instructions allow the processor enough time to commit the instructions.

```
// GROUP 1: set FS in mstatus to 01 initial
asm volatile ( "CSRR t0, mstatus" ); // copy mstatus to t0
asm volatile ( "LI t1, 0xFFFF9FF" ); // create a bitmask for FS
asm volatile ( "AND t0, t0, t1" ); // clear FS bits
asm volatile ( "LI t1, 0x2000" ); // set bitmask for Initial (01)
asm volatile ( "OR t0, t0, t1" ); // set FS to Initial (01)
asm volatile ( "CSRW mstatus, t0" ); // write new mstatus back

// GROUP 2: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 3: write to a floating point register
asm volatile ( "LI a0, 0x3f800000" ); // set value to temporary
register a0
asm volatile ( "FMV.W.X f1, a0" ); // this instruction should change
FS

// GROUP 4: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
```